

Organisation du code et séparation des responsabilités

Objectifs de cette section

À ce stade, votre application devient plus complexe. Il est temps d'apprendre à **structurer votre code** pour le rendre plus clair, maintenable et évolutif. À la fin de cette section, vous serez capable de :

- Identifier les responsabilités dans une application frontend.
 - Organiser votre code en fonctions claires et réutilisables.
 - Distinguer logique métier, manipulation DOM et gestion d'événements.
 - Préparer le terrain pour passer à une approche composants.
-

1. Identifier les responsabilités

Dans une interface interactive, on distingue plusieurs responsabilités :

- **Gestion des données** (ici : la liste des tâches).
- **Manipulation du DOM** (création, suppression, affichage).
- **Gestion des événements** (clics, soumission de formulaires...).

En séparant ces rôles, on évite que tout le code soit entassé dans un seul gros bloc illisible.

2. Créer des fonctions dédiées

Chaque action peut (et doit) être isolée dans une fonction :

- Une fonction pour ajouter une tâche.
- Une fonction pour supprimer une tâche.
- Une fonction pour créer un élément HTML avec les bons attributs.

Exemple : fonction de création de tâche

```
// script.js
function createTaskElement(text) {
  const li = document.createElement("li");
  li.textContent = text;

  const btn = document.createElement("button");
  btn.textContent = "Supprimer";
  btn.addEventListener("click", () => {
    li.remove();
  });

  li.appendChild(btn);
  return li;
}
```

3. Utiliser des fichiers séparés

Lorsque le code grossit, il devient pertinent de :

- Créer un fichier JavaScript distinct pour les **fonctions métier** (ex : `todo.js`).
- Garder un autre fichier pour le **script principal** (`script.js`) qui ne fait qu'orchestrer.

Cela rend votre application plus lisible et facilite la maintenance.

Exemple : séparation du fichier `todo.js`

```
// todo.js
export function createTaskElement(text) {
  const li = document.createElement("li");
  li.textContent = text;

  const btn = document.createElement("button");
  btn.textContent = "Supprimer";
  btn.addEventListener("click", () => {
    li.remove();
  });

  li.appendChild(btn);
  return li;
}
```

```
// script.js
import {createTaskElement} from "./todo.js";

const form = document.getElementById("taskForm");
const input = document.getElementById("taskInput");
const list = document.getElementById("taskList");

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const value = input.value.trim();
  if (value !== "") {
    const taskEl = createTaskElement(value);
    list.appendChild(taskEl);
    input.value = "";
  }
});
```

4. Exercice de mise en pratique

Consignes

1. Reprenez votre projet.
2. Créez un fichier `todo.js` contenant une ou plusieurs fonctions dédiées :
 - `createTaskElement(text)` : crée un élément `<li>` avec le texte et un bouton "Supprimer".
 - (facultatif) `renderTaskList(tasks)` : génère une liste entière.
3. Dans votre `script.js`, importez et utilisez ces fonctions pour gérer l'ajout des tâches.

Organisation attendue

```
/exercices
├── 05-organisation-code
│   ├── index.html
│   ├── script.js
│   └── todo.js
```